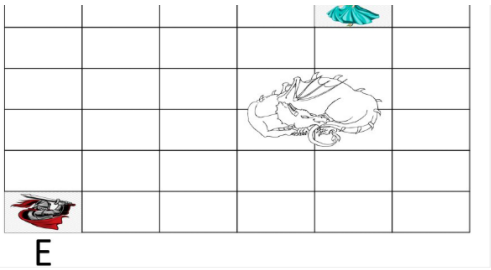
OBIETTIVO



- Modellare il problema:
  - Definizione *PEAS* dell'agente.
  - Definizione delle proprietà dell'ambiente.
- Dare una soluzione in Pseudocodice, definendo le procedure importanti `save(AgPos, DrPos, ExPos, PrPos) -> Path`, dove:
  - `AgPos` è la posizione (X_a, Y_a) iniziale dell'agente
  - `DrPos` è la posizione $((X_1,Y_1), (X_2,Y_2), (X_3,Y_3), (X_4,Y_4))$ del drago
  - `ExPos` è la posizione di entrata/uscita
  - `PrPos` è la posizione (X_p,Y_p) della principessa
  - `Path` è la sequenza di azioni (N,S,E,W,P) che il cavaliere esegue per salvare la principessa

1) P.E.A.S

P → performance → il cavaliere deve salvare la principessa senza svegliare il drago eseguendo il numero minore possibile di mosse (cammino minimo)

E → environment → una matrice con 6x6 = 36 caselle,
1 occupata dal cavaliere [1,1]
4 dal drago [4,4] [5,4] [4,3] [5,3]
1 dalla principessa [5,6]

A → attuatori → Gambe e braccia del cavaliere

S → Sensori → Viste

2) Proprietà ambiente

- Completam. OSS.
- Mono Agente
- Deterministico
- Discreto
- Statico
- Noto
- Sequenziale

3) Modello Ambiente → stati

Possiamo rappresentare l'ambiente come un grafo, bidirezionale e connesso.
Ogni nodo è una casella e gli archi hanno costo 1, tranne quelli che si collegano con le celle del drago che hanno costo $+\infty$.

Ogni nodo rappresenta uno stato dell'ambiente
Ci sono, nella 1° iterazione, 36 stati in cui si può trovare il cav. (da [1,1] verso la princess)
Analogamente altri 36 stati (da [5,6] verso [1,1])

Stato iniziale → Due stati iniziali
- [1,1] → con obiettivo principessa
- [5,6] → con obiettivo uscita

Stati finali → Due stati finali
- [5,6] → prendere principessa
- [1,1] → uscire CON principessa

Azioni → • Su • Giù • Destra • Sinistra • Prendi

Modello di transizione → (descrizione azioni)
- destra → sposta l'agente dalla casella $[x, y]$ a $[x+1, y]$
  $\forall x = 1, \dots, 5$ e $\forall y = 1, \dots, 6$
- sinistra → da $[x, y]$ a $[x-1, y]$
- su → da $[x, y]$ a $[x, y+1]$
  $\forall x = 1, \dots, 6$ e $\forall y = 1, \dots, 5$
- giù → da $[x, y]$ a $[x, y-1]$
- prendi → "prende" la principessa appena arriva al primo stato obiettivo

Costo azioni → Costo 1 (drago = $+\infty$)

Il codice (tua versione)

```
function RUN-EVAL-ENVIR (state, UPDATE-FN, CV, PERFORMANCE-FN)
returns CE
  local variables  CE   % inizialmente 0

  repeat
      (NWpos, Dpos, PRpos, EXpos) ← GET-PERCEPT(CV, state)

      Action ← CV-PROG(NWpos, Dpos, PRpos, EXpos)

      state  ← UPDATE-FN(state, Action)

      CE     ← PERFORMANCE-FN(state, CE)

  until TERMINATION(state)

  return CE
```

Cosa significa ogni pezzo

Intestazione

- `RUN-EVAL-ENVIR(...)`
  È **il ciclo di vita dell'ambiente**: simula passo-passo l'interazione tra **agente** (il cavaliere) e **ambiente** (la griglia con drago e principessa).
- **Parametri**:
  - `state`: lo **stato corrente dell'ambiente** (posizioni di cavaliere, drago, principessa; celle pericolose; obiettivo attuale, ecc.).
  - `UPDATE-FN`: funzione che **applica gli effetti dell'azione** scelta (sposta il cavaliere, aggiorna goal raggiunti, ecc.).

    - `CV`: l'**agente cavaliere** (serve solo per passarlo a `GET-PERCEPT` o se vuoi più agenti).
    - `PERFORMANCE-FN`: funzione che **aggiorna il punteggio/costo cumulativo** (ad es. +1 per ogni passo, +∞ se entra dal drago, bonus quando raggiunge la principessa, ecc.).
- `returns CE`: alla fine restituisci il **costo/score totale** del comportamento dell'agente.

Variabile locale

- `CE`
  Accumula la **valutazione** (costo o punteggio). Tipicamente parte da 0.

  > Se stai **minimizzando il cammino**, `CE` è la **somma dei costi** delle mosse già fatte.

Il ciclo principale

1) Percezione

`(NWpos, Dpos, PRpos, EXpos) ← GET-PERCEPT(CV, state)`

- `GET-PERCEPT` legge dal `state` **ciò che il cavaliere "vede"**:
    - `NWpos` = posizione **Now**: coordinate correnti del cavaliere (es. `(x,y)`).
    - `Dpos` = posizioni **proibite** legate al drago (la cella del drago e, se vuoi, tutte le adiacenti da evitare).
    - `PRpos` = posizione della **principessa** (primo goal).
    - `EXpos` = **posizione finale** (secondo goal: tornare alla partenza).
- In un ambiente **completamente osservabile**, `GET-PERCEPT` può semplicemente estrarre questi dati dallo stato.

> Nel tuo esercizio: “occhi del cavaliere” (S) = questa funzione.

---

2) Decisione dell’agente (policy/program)

`Action ← CV-PROG(NWpos, Dpos, PRpos, EXpos)`

- `CV-PROG` è **il cervello del cavaliere**: riceve la percezione e **sceglie un’azione** tra {N, S, E, O}.
- Dentro `CV-PROG` in pratica richiami una **ricerca di cammino minimo sul grafo** della griglia:
  - **fase 1**: dall’attuale `NWpos` alla `PRpos`, evitando `Dpos` (archi/celle vietati o costo ∞);
  - **fase 2**: dalla `PRpos` a `EXpos` (ritorno alla base), sempre evitando `Dpos`.
- Se stai eseguendo tutto in **un solo episodio**, `CV-PROG` può:
  - mantenere internamente il **piano** (lista di mosse) e restituire **la prossima mossa**,
  - oppure ricalcolare ad ogni passo il **prossimo passo del cammino minimo** (A*, Dijkstra) verso il **goal corrente**.

> Nel tuo schema: “gambe del cavaliere” (A) = l’azione calcolata qui.

---

3) Aggiornamento dello stato

`state ← UPDATE-FN(state, Action)`

- Applica **l’effetto dell’azione** al mondo:
  - sposta il cavaliere nella cella successiva;
  - se entra nella cella della principessa, **marca “goal1 raggiunto”** e cambia il **goal corrente** a `EXpos`;
  - se esce dai limiti o va in `Dpos`, puoi segnare **fallimento** o **costo infinito**.

- Se vuoi dinamica, qui potresti anche spostare il drago (non necessario nel tuo esercizio).

> È il blocco che "fa avanzare" la simulazione.

---

4) Aggiornamento dello score/costo

`CE ← PERFORMANCE-FN(state, CE)`

- Aggiorna il **costo cumulativo**:
  - +1 (o il peso dell'arco) per ogni passo valido,
  - +B (bonus) quando raggiungi la principessa (opzionale),
  - +∞ o **terminazione** se entri in cella vietata,
  - alla fine puoi anche restituire la **lunghezza totale del cammino minimo**.

> Se i tuoi archi hanno **peso 1** (movimenti unitari) e le celle del drago/adiacenti hanno **peso ∞**, questa funzione è semplicissima.

---

5) Condizione di arresto

`until TERMINATION(state)`

Arresta quando:

- **hai raggiunto i due goal in sequenza** (prima principessa, poi base), **oppure**
- non ci sono mosse legali (bloccato), **oppure**
- è stato raggiunto un **limite di passi** (per sicurezza nelle simulazioni).

---

6) Ritorno del risultato

`return CE`

- Restituisci il costo totale del percorso trovato.
- In un compito d'esame, spesso vogliono proprio questo numero (o il percorso se lo memorizzi).

⚙️ Codice (ripulito e riscritto)

```
function CV-PROGRAM(percept) returns action
persistent:
    plan      % sequenza di azioni, inizialmente vuota
    state     % stato corrente del mondo
    goal      % nodo goal
    problem   % problema corrente

if plan is EMPTY then
    NG ← FORMULATE-GOAL(EXPOS, DPOS, PRPOS)
    problem ← FORMULATE-PROBLEM(DPOS, EXPOS, OCPOS, PRPOS, NG)
    plan ← SEARCH(problem)         % es. A* con f(n) = g(n) +
h(n)
    if plan = failure then
        return null-action
else
    if NWPOS = PRPOS then
        catch                     % ha raggiunto la principessa,
cambia goal
    else if NG = EXPOS then
        exit                      % ha completato il secondo
obiettivo
end if

action ← FIRST(plan)
plan ← REST(plan)
return action
```

🔍 Significato riga per riga

🧠 `persistent: ...`

Queste sono **variabili persistenti** → mantengono il loro valore tra una chiamata e la successiva.
L'agente non "ricomincia da zero" a ogni percezione, ma **ricorda**:

- `plan`: il piano corrente (lista ordinata di azioni da eseguire);
- `state`: lo stato attuale del mondo;
- `goal`: il target (prima la principessa, poi la base);
- `problem`: il modello di ricerca associato a quel goal.

📌 In pratica: serve per fare **pianificazione incrementale** → l'agente calcola un piano una volta sola, poi lo esegue passo per passo.

🔹 `if plan is EMPTY then`

Controlla se **non esiste un piano attivo**.

- All'inizio dell'esecuzione, `plan` è vuoto → bisogna **formulare un obiettivo** e un **problema di ricerca**, e poi **calcolare un piano**.

🧭 `NG ← FORMULATE-GOAL(EXPOS, DPOS, PRPOS)`

Qui l'agente decide **dove deve andare** in questa fase:

- `EXPOS` = posizione iniziale (dove tornerà dopo),
- `PRPOS` = posizione della principessa (primo goal),
- `DPOS` = posizione del drago (da evitare).

➡️ Il goal iniziale è:
raggiungere la principessa evitando il drago.

Dopo averla raggiunta, il prossimo goal sarà:
tornare alla base (`EXPOS`) evitando il drago.

> Quindi questa funzione serve per **scegliere il nuovo obiettivo**, a seconda di dove si trova il cavaliere.

🧩 `problem ← FORMULATE-PROBLEM(...)`

Qui costruisce il **problema di ricerca**:

- Stato iniziale = posizione corrente (`NWPOS`),
- Goal = `NG` (il target scelto sopra),
- Azioni = mosse possibili (N, S, E, O),
- Transizioni = movimenti consentiti nella griglia,
- Costo = 1 per ogni passo, ∞ se il movimento entra in una cella del drago.

➡️ In pratica: si definisce il **grafo della griglia**, con nodi, archi e costi.

🧮 `plan ← SEARCH(problem)`

L'agente usa un **algoritmo di ricerca** (es. A*) per trovare il percorso migliore:

- `SEARCH(problem)` = `A*(problem, f(n)=g(n)+h(n))`
- `g(n)` = costo del cammino fatto (numero di passi),
- `h(n)` = stima euristica (distanza di Manhattan fino al goal).

Se la ricerca ha successo, `plan` sarà una lista di azioni tipo:

`[N, N, E, E, E, N, N]`

Se fallisce (`plan = failure`), l'agente non può agire → restituisce `null action`.

⚠️ Caso 2 – `else` (il piano esiste già)

Se c'è un piano, l'agente non pianifica di nuovo.
Si limita a eseguire il prossimo passo e a controllare se deve cambiare obiettivo.

`if NWPOS = PRPOS then catch`

- Significa che ha raggiunto la principessa.
  Quindi cambia il goal: il nuovo obiettivo (`NG`) diventa la posizione iniziale (`EXPOS`).

`else if NG = EXPOS then exit`

- Ha raggiunto anche la posizione iniziale → fine del compito, terminazione.

🚶 `action ← FIRST(plan)`

- Prende la prima azione del piano (es. "muoviti a nord").

📜 `plan ← REST(plan)`

- Elimina quella azione dal piano → così la prossima volta eseguirà la successiva.

🔁 `return action`

- Restituisce l'azione scelta, che verrà poi eseguita nel ciclo principale (`RUN-EVAL-ENVIR`).

🔄 Ciclo completo (riassunto del comportamento)

1. Percepisce l'ambiente (posizioni di cavaliere, drago, principessa).
2. Se non ha un piano,
   - definisce il goal,
   - costruisce il problema,

- calcola il **cammino minimo** con A*.

3. Se **ha già un piano**,
   - esegue la **prossima azione**,
   - controlla se deve cambiare goal (ha preso la principessa → torna indietro).
4. Restituisce l'**azione da eseguire ora**.
5. Alla chiamata successiva, il piano viene aggiornato e continua.